

프로그래밍 역량 강화 전문기관, 민코딩

Restaurant Booking



목차

1. Restaurant Booking KATA 소개
2. 테스트 케이스 작성 (with Mocking)

Legacy Code Refactoring

Restaurant Booking KATA 소개

레스토랑

✓ 레스토랑에서 예약 완료 처리 해주는 프로그램

1. 고객의 예약을 받는다. (사람수, 시간)
2. 예약 스케줄에 추가한다.
3. 예약에 문제가 생기면 Exception을 발생시킨다.
4. 예약에 아무런 문제가 없으면, 고객에게 SMS를 보낸다.
5. 고객이 email도 가지고 있다면, 고객에게 email을 또 보낸다.



레스토랑 규칙

✓ 예약 규칙 (규칙을 어기면, Exception 발생)

1. 정시에만 예약을 받는다.
 - 09:03 예약이 들어오면 예외발생 (09:00은 가능)
2. 시간당 최대 수용인원이 정해져 있다.
 - 특정 시간에, 수용인원보다 더 많은 사람이 예약이 들어오면 예외 발생
3. 일요일에는 예약을 받지 않는다.



예약자 규칙

✓ 예약자 정보

- 이름, 전화번호는 필수이다.
- email은 옵션이다. (email을 주면, 예약 결과를 email로 받을 수 있다.)

✓ 스케줄 정보

- 예약자명
- 예약하고 싶은 날짜/시간
- 예약 인원 수



프로젝트 목적

✓ 레거시 코드를 리팩토링 하는 목적이 아니다.

- Mocking 과 함께 유닛테스트를 작성하고,
유닛테스트를 리팩토링 한다.

✓ 프로젝트 목적

1. 유닛테스트를 작성한다. 필요하면 Test Double을 직접 제작한다.
2. (옵션)작성한 유닛테스트를 Mocking Library를 사용하는 것으로 리팩토링 한다.

소스코드 준비

사내 링크

- <https://github.samsungds.net/cra1-sec/RestaurantBooking>

사외 링크

- <https://github.com/mincoding1/RestaurantBooking>

[참고] C++ 에서 날짜 + 시간 관리 샘플코드

- <https://gist.github.com/mincoding1/9a50ee4c244fa51fd984a574906f8ca7>

소스코드 분석 1

예약자 정보 : 이름, 전화번호, 이메일

스케줄 정보 : 예약자 시간, 예약 인원수, **예약자 정보**

```
#pragma once
#include <string>

using std::string;

class Customer {
public:
    Customer(string name, string phoneNumber) :
        name{ name },
        phoneNumber{ phoneNumber } {
    }

    Customer(string name, string phoneNumber, string email) :
        name{ name },
        phoneNumber{ phoneNumber },
        email{ email } {
    }

    string getEmail() {
        return email;
    }

private:
    string name;
    string phoneNumber;
    string email;
};
```

Customer.cpp

```
#pragma once
#include <ctime>
#include "Customer.hpp"

using std::time_t;

class Schedule {
public:
    Schedule(tm dateTime, int numberOfPeople, Customer customer) :
        dateTime{ dateTime },
        numberOfPeople{ numberOfPeople },
        customer{ customer } {
    }

    tm getDateTime() {
        return dateTime;
    }

    int getNumberOfPeople() {
        return numberOfPeople;
    }

    Customer getCustomer() {
        return customer;
    }

private:
    tm dateTime;
    int numberOfPeople;
    Customer customer;
};
```

Schedule.cpp

소스코드 분석 2

- ✓ SMS / email 전송할 수 있는 객체
 - 구현이 완료되어 있다고 가정한다

SmsSender.cpp

```
#pragma once
#include "Schedule.cpp"

class SmsSender {
public:
    void send(Schedule *schedule) {
        //send message to PhoneNumber
    }
};
```

MailSender.cpp

```
#pragma once
#include "Schedule.cpp"

class MailSender {
public:
    void sendMail(Schedule *schedule) {
        //send mail
    }
};
```

소스코드 분석 3

✓ 예약을 받는 코드

- addSchedule() 메서드

주석처리된 코드
PC 시간기준,
일요일 실행시
Exception 발생

BookingScheduler.cp

```
class BookingScheduler {
public:
    BookingScheduler(int capacityPerHour) :
        capacityPerHour{ capacityPerHour } {
        smsSender = new SmsSender();
        mailSender = new MailSender();
    }

    void addSchedule(Schedule* schedule) {

        // 정각에 예약하지 않을 경우 RuntimeException 발생
        if (schedule->getDateTime().tm_min != 0) {
            throw std::runtime_error("Booking should be on the hour.");
        }

        // 시간당 예약인원을 초과할 경우 RuntimeException 발생
        int numberOfPeople = schedule->getNumberOfPeople();
        for (Schedule* bookedSchedule : schedules) {
            if (isSameTime(bookedSchedule->getDateTime(), schedule->getDateTime())) {
                numberOfPeople += bookedSchedule->getNumberOfPeople();
            }
        }
        if (numberOfPeople > capacityPerHour) {
            throw std::runtime_error("Number of people is over restaurant capacity per hour");
        }

        /*
        // 일요일에는 시스템을 오픈하지 않는다.
        time_t now = time(nullptr);
        if (getDayOfWeek(now) == "Sunday") {
            throw std::runtime_error("Booking system is not available on sunday");
        }
        */

        schedules.push_back(schedule);

        // 고객에게 SMS 발송
        smsSender->send(schedule);
        // 고객이 E Mail을 가지고 있을 경우 E Mail 발송
        if (schedule->getCustomer().getEmail() != "") {
            mailSender->sendMail(schedule);
        }
    }
}
```

Unit Test

- ✓ 한글로 되어있음 (Test Case를 한글로 작성을 허용하는 팀도 존재)
 - 헤더 파일 경로를 본인 프로젝트 경로에 맞게 수정할 것.

```
#include "pch.h"
#include "../Project20/BookingScheduler.cpp"

TEST(BookingSchedulerTest, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {
}

TEST(BookingSchedulerTest, 예약은_정시에만_가능하다_정시인_경우_예약가능) {
}

TEST(BookingSchedulerTest, 시간대별_인원제한이_있다_같은_시간대에_Capacity_초과할_경우_예외발생) {
}

TEST(BookingSchedulerTest, 시간대별_인원제한이_있다_같은_시간대가_다르면_Capacity_차있어도_스케줄_추가_성공) {
}

TEST(BookingSchedulerTest, 예약완료시_SMS는_무조건_발송) {
}

TEST(BookingSchedulerTest, 이메일이_없는_경우에는_이메일_미발송) {
}

TEST(BookingSchedulerTest, 이메일이_있는_경우에는_이메일_발송) {
}
```

제출 준비

✓ Repository 이름

- RestaurantBooking-사번
- git push 하기

✓ Collaborator

- 팀원 & 강사

✓ Branch Protection Rule

- Approve 1 명 필수

✓ Branch명

- 1 단계 : feature/mocking_step1
- 2 단계 : feature/mocking_step2

필수 PR 횟수 : 2회

유닛 테스트를 작성한다. 필요시 Test Double을 직접 제작한다.

1 단계 : 테스트 케이스 작성 (with Mocking)

테스트 케이스 작성

✓ Step1. "예약은 정시에만 가능하다." 테스트 작성 1

- 2021.3.26 의 09:05로 예약하여 Exception 발생 테스트
- 시간 관련 문법은 우측 링크를 통해 이해한다. <https://gist.github.com/mincoding1/9a50ee4c244fa51fd984a574906f8ca7>

```
TEST(BookingSchedulerTest, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {  
    //arrange  
    tm notOnTheHour = { 0 };  
    notOnTheHour.tm_year = 2021 - 1900; //-1900 : tm 구조체는 1900년도 이후 데이터로 기록  
    notOnTheHour.tm_mon = 03 - 1; //-1 : tm 구조체는 month를 0 부터 관리  
    notOnTheHour.tm_mday = 26;  
    notOnTheHour.tm_hour = 9; //9시  
    notOnTheHour.tm_min = 5; //5분, 정각이 아님  
    notOnTheHour.tm_isdst = -1; //-1 : daylight를 자동 설정으로 맡김  
    mktime(&notOnTheHour); //나머지 tm 구조체 멤버 값에 대해 자동으로 계산하여 채움.  
  
    Customer customer{ "Fake name", "010-1234-5678" };  
    Schedule *schedule = new Schedule{ notOnTheHour, 1, customer };  
    BookingScheduler bookingScheduler{ 3 };  
  
    //act  
    EXPECT_THROW({  
        bookingScheduler.addSchedule(schedule);  
    }, std::runtime_error);  
  
    //assert  
    //expected runtime exception  
}
```

테스트 케이스 작성

✓ Step2. "예약은 정시에만 가능하다." 테스트 작성 2

- 2021.3.26 의 09:00로 예약하여 정상 예약 확인

```
TEST(BookingSchedulerTest, 예약은_정시에만_가능하다_정시인_경우_예약가능) {  
    //arrange  
    tm onTheHour = { 0 };  
    onTheHour.tm_year = 2021 - 1900;  
    onTheHour.tm_mon = 03 - 1;  
    onTheHour.tm_mday = 26;  
    onTheHour.tm_hour = 9;  
    onTheHour.tm_min = 0; //정각  
    onTheHour.tm_isdst = -1;  
    mktime(&onTheHour);  
  
    Customer customer{ "Fake name", "010-1234-5678" };  
    Schedule* schedule = new Schedule{ onTheHour, 1, customer };  
    BookingScheduler bookingScheduler{ 3 };  
  
    //act  
    bookingScheduler.addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(true, bookingScheduler.hasSchedule(schedule));  
}
```

테스트 케이스 작성

✓ Step3-1. Test Fixture 만들기

- 중복코드를 제거하기 위해, Test Fixture를 만들어둔다.
- tm 구조체를 만들어주는 getTime 함수를 추가한다.

```
#include "pch.h"
#include "../Project20/BookingScheduler.cpp"

class BookingItem : public testing::Test {
public:
    tm getTime(int year, int mon, int day, int hour, int min) {
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };
        mktime(&result);
        return result;
    }
};

TEST_F(BookingItem, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {
    //arrange
    tm notOnTheHour = { 0 };
```

Test Fixture

모든 Test Case에 (9개)
이 Fixture를 적용한다.

테스트 케이스 작성

✓ Step3-2. 중복코드 제거

- 관심사가 아닌 부분은, 캡슐화를 통해 추상 시킨다.

```
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {  
    //arrange  
    tm notOnTheHour = { 0 };  
    notOnTheHour.tm_year = 2021 - 1900; //-1900 : tm 구조체는 1900년도 이후 데이터로 기록  
    notOnTheHour.tm_mon = 03 - 1; //-1 : tm 구조체는 month를 0 부터 관리  
    notOnTheHour.tm_mday = 26;  
    notOnTheHour.tm_hour = 9; //9시  
    notOnTheHour.tm_min = 5; //5분, 정각이 아님  
    notOnTheHour.tm_isdst = -1; //-1 : daylight를 자동 설정으로 맡김  
    mktime(&notOnTheHour); //나머지 tm 구조체 멤버 값에 대해 자동으로 계산하여 채움.  
  
    Customer customer{ "Fake name", "010-1234-5678" };  
    Schedule *schedule = new Schedule{ notOnTheHour, 1, customer };  
    BookingScheduler bookingScheduler{ 3 };  
  
    //act  
    EXPECT_THROW({  
        bookingScheduler.addSchedule(schedule);  
    }, std::runtime_error);  
  
    //assert  
    //expected runtime exception  
}  
  
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시인_경우_예약가능) {  
    //arrange  
    tm onTheHour = { 0 };  
    onTheHour.tm_year = 2021 - 1900;  
    onTheHour.tm_mon = 03 - 1;  
    onTheHour.tm_mday = 26;  
    onTheHour.tm_hour = 9;  
    onTheHour.tm_min = 0; //정각  
    onTheHour.tm_isdst = -1;  
    mktime(&onTheHour);
```



```
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {  
    //arrange  
    tm notOnTheHour = getTime(2021, 3, 26, 9, 5);  
    Customer customer{ "Fake name", "010-1234-5678" };  
    Schedule *schedule = new Schedule{ notOnTheHour, 1, customer };  
    BookingScheduler bookingScheduler{ 3 };  
  
    //act  
    EXPECT_THROW({  
        bookingScheduler.addSchedule(schedule);  
    }, std::runtime_error);  
  
    //assert  
    //expected runtime exception  
}  
  
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시인_경우_예약가능) {  
    //arrange  
    tm onTheHour = getTime(2021, 3, 26, 9, 0);
```

테스트 케이스 작성

✓ Step3-3. 자주 사용할 코드를 Fixture로 옮기기

- time, customer 객체를 Fixture로 옮긴다.

```
class BookingItem : public testing::Test {
public:
    tm getTime(int year, int mon, int day, int hour, int min) {
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };
        mktime(&result);
        return result;
    }
};
```

```
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {
    //arrange
    tm notOnTheHour = getTime(2021, 3, 26, 9, 5);
    Customer customer{ "Fake name", "010-1234-5678" };
    Schedule *schedule = new Schedule{ notOnTheHour, 1, customer };
    BookingScheduler bookingScheduler{ 3 };

    //act
    EXPECT_THROW({
        bookingScheduler.addSchedule(schedule);
    }, std::runtime_error);

    //assert
    //expected runtime exception
}
```

```
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시인_경우_예약가능) {
    //arrange
    tm onTheHour = getTime(2021, 3, 26, 9, 0);

    Customer customer{ "Fake name", "010-1234-5678" };
    Schedule* schedule = new Schedule{ onTheHour, 1, customer };
    BookingScheduler bookingScheduler{ 3 };
}
```



```
class BookingItem : public testing::Test {
protected:
    void SetUp() override {
        notOnTheHour = getTime(2021, 3, 26, 9, 5);
        onTheHour = getTime(2021, 3, 26, 9, 0);
    }
public:
    tm getTime(int year, int mon, int day, int hour, int min) {
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };
        mktime(&result);
        return result;
    }

    tm notOnTheHour;
    tm onTheHour;
    Customer customer{ "Fake name", "010-1234-5678" };
};
```

```
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {
    //arrange
    Schedule *schedule = new Schedule{ notOnTheHour, 1, customer };
    BookingScheduler bookingScheduler{ 3 };

    //act
    EXPECT_THROW({
        bookingScheduler.addSchedule(schedule);
    }, std::runtime_error);

    //assert
    //expected runtime exception
}
```

```
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시인_경우_예약가능) {
    //arrange
    Schedule* schedule = new Schedule{ onTheHour, 1, customer };
    BookingScheduler bookingScheduler{ 3 };
}
```

테스트 케이스 작성

✓ Step3-4. Magic Number로 작성된 필드 이름 변경

- onTheHour □ ON_THE_HOUR
- notOnTheHour □ NOT_ON_THE_HOUR
- customer □ CUSTOMER

```
class BookingItem : public testing::Test {
protected:
    void SetUp() override {
        NOT_ON_THE_HOUR = getTime(2021, 3, 26, 9, 5);
        ON_THE_HOUR = getTime(2021, 3, 26, 9, 0);
    }

public:
    tm getTime(int year, int mon, int day, int hour, int min) {
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };
        mktime(&result);
        return result;
    }

    tm NOT_ON_THE_HOUR;
    tm ON_THE_HOUR;
    Customer CUSTOMER{ "Fake name", "010-1234-5678" };
};
```

테스트 케이스 작성

✓ Step3-5. Magic number 치환

- 숫자 1 (numberOfPeople)을 UNDER_CAPACITY 로 변경
- 숫자 3 (capacityPerHour)을 CAPACITY_PER_HOUR로 변경

```
tm NOT_ON_THE_HOUR;
tm ON_THE_HOUR;
Customer CUSTOMER{ "Fake name", "010-1234-5678" };
};

TEST_F(BookingItem, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {
    //arrange
    Schedule *schedule = new Schedule{ NOT_ON_THE_HOUR, 1, CUSTOMER };
    BookingScheduler bookingScheduler{ 3 };

    //act
    EXPECT_THROW({
        bookingScheduler.addSchedule(schedule);
    }, std::runtime_error);

    //assert
    //expected runtime exception
}

TEST_F(BookingItem, 예약은_정시에만_가능하다_정시인_경우_예약가능) {
    //arrange
    Schedule* schedule = new Schedule{ ON_THE_HOUR, 1, CUSTOMER };
    BookingScheduler bookingScheduler{ 3 };
}
```



```
tm NOT_ON_THE_HOUR;
tm ON_THE_HOUR;
Customer CUSTOMER{ "Fake name", "010-1234-5678" };
const int UNDER_CAPACITY = 1;
const int CAPACITY_PER_HOUR = 3;
};

TEST_F(BookingItem, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {
    //arrange
    Schedule *schedule = new Schedule{ NOT_ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };
    BookingScheduler bookingScheduler{ CAPACITY_PER_HOUR };

    //act
    EXPECT_THROW({
        bookingScheduler.addSchedule(schedule);
    }, std::runtime_error);

    //assert
    //expected runtime exception
}

TEST_F(BookingItem, 예약은_정시에만_가능하다_정시인_경우_예약가능) {
    //arrange
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };
    BookingScheduler bookingScheduler{ CAPACITY_PER_HOUR };

    //act
    bookingScheduler.addSchedule(schedule);
}
```

테스트 케이스 작성

✓ Step3-6. 중복코드 제거

- bookingScheduler를 Fixture로 이동

```
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {  
    //arrange  
    Schedule *schedule = new Schedule{ NOT_ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };  
    BookingScheduler bookingScheduler{ CAPACITY_PER_HOUR };  
  
    //act  
    EXPECT_THROW({  
        bookingScheduler.addSchedule(schedule);  
    }, std::runtime_error);  
  
    //assert  
    //expected runtime exception  
}  
  
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시인_경우_예약가능) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };  
    BookingScheduler bookingScheduler{ CAPACITY_PER_HOUR };  
  
    //act  
    bookingScheduler.addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(true, bookingScheduler.hasSchedule(schedule));  
}
```



```
class BookingItem : public testing::Test {  
protected:  
    void SetUp() override {  
        NOT_ON_THE_HOUR = getTime(2021, 3, 26, 9, 5);  
        ON_THE_HOUR = getTime(2021, 3, 26, 9, 0);  
    }  
  
public:  
    tm getTime(int year, int mon, int day, int hour, int min) {  
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };  
        mktime(&result);  
        return result;  
    }  
  
    tm NOT_ON_THE_HOUR;  
    tm ON_THE_HOUR;  
    Customer CUSTOMER{ "Fake name", "010-1234-5678" };  
    const int UNDER_CAPACITY = 1;  
    const int CAPACITY_PER_HOUR = 3;  
  
    BookingScheduler bookingScheduler{ CAPACITY_PER_HOUR };  
};  
  
TEST_F(BookingItem, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {  
    //arrange  
    Schedule *schedule = new Schedule{ NOT_ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };  
  
    //act  
    EXPECT_THROW({  
        bookingScheduler.addSchedule(schedule);  
    }, std::runtime_error);  
}
```

테스트 케이스 작성

✓ Step3-7. 결과 확인

- 지금까지 변경한 결과가
우측과 동일한지 확인한다.
- 시간당 최대 3인 제한으로
세팅한 것을 기억하자.

```
#include "pch.h"
#include "../Project20/BookingScheduler.cpp"

class BookingItem : public testing::Test {
protected:
    void SetUp() override {
        NOT_ON_THE_HOUR = getTime(2021, 3, 26, 9, 5);
        ON_THE_HOUR = getTime(2021, 3, 26, 9, 0);
    }

public:
    tm getTime(int year, int mon, int day, int hour, int min) {
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };
        mktime(&result);
        return result;
    }

    tm NOT_ON_THE_HOUR;
    tm ON_THE_HOUR;
    Customer CUSTOMER{ "Fake name", "010-1234-5678" };
    const int UNDER_CAPACITY = 1;
    const int CAPACITY_PER_HOUR = 3;

    BookingScheduler bookingScheduler{ CAPACITY_PER_HOUR };
};

TEST_F(BookingItem, 예약은_정시에만_가능하다_정시가_아닌경우_예약불가) {
    //arrange
    Schedule* schedule = new Schedule{ NOT_ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };

    //act
    EXPECT_THROW({
        bookingScheduler.addSchedule(schedule);
    }, std::runtime_error);

    //assert
    //expected runtime exception
}

TEST_F(BookingItem, 예약은_정시에만_가능하다_정시인_경우_예약가능) {
    //arrange
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };

    //act
    bookingScheduler.addSchedule(schedule);

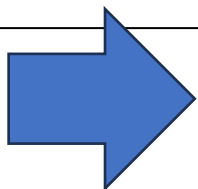
    //assert
    EXPECT_EQ(true, bookingScheduler.hasSchedule(schedule));
}
```

테스트 케이스 작성

✓ Step4. 최대 수용 인원을 초과했을 때, 예외 발생 테스트

- 3인 스케줄을 추가한다.
- 동시간에 1인 스케줄 하나 더 추가했을 때 예외 발생 테스트

```
TEST_F(BookingItem, 시간대별_인원제한이_있다_같은_시간대에_Capacity_초과할_경우_예외발생) {  
}
```



```
TEST_F(BookingItem, 시간대별_인원제한이_있다_같은_시간대에_Capacity_초과할_경우_예외발생) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, CAPACITY_PER_HOUR, CUSTOMER };  
    bookingScheduler.addSchedule(schedule);  
  
    //act  
    try {  
        Schedule* newSchedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };  
        bookingScheduler.addSchedule(newSchedule);  
        FAIL(); //Exception이 발생할 예정이기에, FAIL() 함수에 도달하지 않는다.  
    }  
    catch (std::runtime_error& e) {  
        //assert  
        EXPECT_EQ(string{ e.what() }, string{ "Number of people is over restaurant capacity per hour" });  
    }  
}
```

테스트 케이스 작성

✓ Step5-1. 인원제한을 채우고, 다른 시간대에 예약 걸면 정상임을 확인하기

- 3인 스케줄을 추가한다.
- ON_THE_HOUR 시간 + 1 시간으로 예약했을 때 정상임을 확인한다.

```
TEST_F(BookingItem, 시간대별_인원제한이_있따_같은_시간대가_다르면_Capacity_차있어도_스케줄_추가_성공) {  
    //arange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, CAPACITY_PER_HOUR, CUSTOMER };  
    bookingScheduler.addSchedule(schedule);  
  
    //act  
    tm differentHour = ON_THE_HOUR;  
    differentHour.tm_hour += 1;  
    mktime(&differentHour);  
    Schedule* newSchedule = new Schedule{ differentHour, UNDER_CAPACITY, CUSTOMER };  
    bookingScheduler.addSchedule(newSchedule);  
  
    //assert  
    EXPECT_EQ(true, bookingScheduler.hasSchedule(schedule));  
    EXPECT_EQ(true, bookingScheduler.hasSchedule(newSchedule));  
}
```

테스트 케이스 작성

✓ Step5-2. 관심사가 아닌 코드는 추상화 시킨다.

- plusHour 함수를 제작하여 리팩토링을 한다.

```
class BookingItem : public testing::Test {
protected:
    void SetUp() override {
        NOT_ON_THE_HOUR = getTime(2021, 3, 26, 9, 5);
        ON_THE_HOUR = getTime(2021, 3, 26, 9, 0);
    }
public:
    tm getTime(int year, int mon, int day, int hour, int min) {
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };
        mktime(&result);
        return result;
    }

    tm plusHour(tm base, int hour) {
        base.tm_hour += hour;
        mktime(&base);
        return base;
    }
};
```

Fixture에
함수 추가

```
TEST_F(BookingItem, 시간대별_인원제한이_있다_같은_시간대가_다른_시간대_보다_인원제한이_높다) {
    //arrange
    Schedule* schedule = new Schedule{ ON_THE_HOUR, CAPACITY_P };
    bookingScheduler.addSchedule(schedule);

    //act
    tm differentHour = plusHour(ON_THE_HOUR, 1);
    Schedule* newSchedule = new Schedule{ differentHour, UNDER_CAPACITY };
    bookingScheduler.addSchedule(newSchedule);

    //assert
    EXPECT_EQ(true, bookingScheduler.hasSchedule(schedule));
}
```

기존 코드를 추상화 시킴

테스트 케이스 작성

✓ Step6. SMS 테스트

- 예약완료시 SMS가 전송되는지 Unit Test를 작성하고자 한다.
- 하지만, 테스트를 할 때마다 SMS를 보내게 되면 비용이 발생하게 된다.
 - Test Double 필요!
- Mocking Library를 사용하지 않고, SMS 대역(Double)을 직접 만들어보자.

테스트 케이스 작성

✓ Step6-1. SmsSender에 **virtual** 키워드 추가하기

- Overriding하는 가짜 객체를 생성하기 위해 준비 작업이 필요하다.
- 기존 SmsSender 클래스에 send 함수를 Overriding 하기 위해 virtual 키워드를 붙여준다.

```
#pragma once
#include "Schedule.cpp"

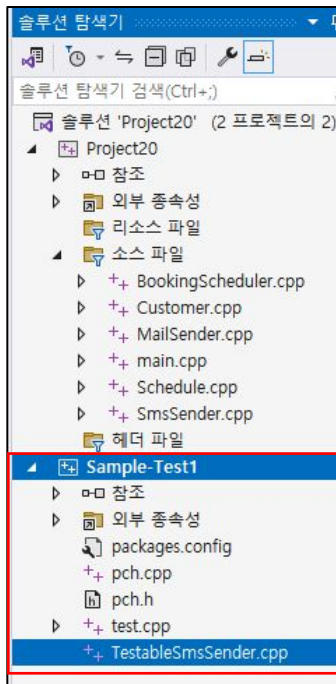
class SmsSender {
public:
    virtual void send(Schedule *schedule) {
        //send message to PhoneNumber
    }
};
```

Overriding을 위해 virtual을 붙여준다.

테스트 케이스 작성

✓ Step6-2. SMS Test Double 만들기

- 실제 객체(SmsSender)을 상속 받은 후, Overriding을 하여 가짜 객체를 생성한다.
- Behavior 검증을 위해, flag 변수를 하나 추가한다.



Test 전용 객체이므로,
구글테스트 프로젝트에 파일을 생성

```
#include <iostream>
#include "pch.h"
#include "../Project20/SmsSender.cpp"

class TestableSmsSender : public SmsSender {
public:
    void send(Schedule* schedule) override {
        std::cout << "테스트용 SmsSender class의 send메서드 실행됨\n";
        sendMethodIsCalled = true;
    }

    bool isSendMethodIsCalled() {
        return sendMethodIsCalled;
    }

private:
    bool sendMethodIsCalled;
};
```

TestableSmsSender.cp

Mocking을 위해, 신규 테스트용 Class를 제작한다.

테스트 케이스 작성

✓ Step6-3. Test Double로 Unit Test하기

- SmsSender 대신, 제작한 Test Double (TestableSmsSender) 클래스를 사용.
- 예약완료시, sender 메서드를 정상적으로 호출 하는지 Double로 테스트 가능

```
#include "pch.h"
#include "../Project20/BookingScheduler.cpp"
#include "TestableSmsSender.cpp"
```

test.cpp 에 헤더파일 추가하기

```
TEST_F(BookingItem, 예약완료시_SMS는_무조건_발송) {
    //arrange
    TestableSmsSender testableSmsSender;
    Schedule* schedule = new Schedule{ ON_THE_HOUR, CAPACITY_PER_HOUR, CUSTOMER };
    bookingScheduler.setSmsSender(&testableSmsSender);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    EXPECT_EQ(true, testableSmsSender.isSendMethodIsCalled());
}
```

호출된 적이 있어야 PASS 이다.

테스트 케이스 작성

✓ Step7-1. testableSmsSender 를 Fixture로 옮기기

- 자주 사용될 수 있도록,
Fixture 코드로 이동시킨다.

```
TEST_F(BookingItem, 예약완료시_SMS는_무조건_발송) {  
    //arrange  
    TestableSmsSender testableSmsSender;  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, CAPACITY_PER_HOUR };  
    bookingScheduler.setSmsSender(&testableSmsSender);  
  
    //act  
    bookingScheduler.addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(true, testableSmsSender.isSendMethodIsCalled());  
}
```



```
class BookingItem : public testing::Test {  
protected:  
    void SetUp() override {  
        NOT_ON_THE_HOUR = getTime(2021, 3, 26, 9, 5);  
        ON_THE_HOUR = getTime(2021, 3, 26, 9, 0);  
  
        bookingScheduler.setSmsSender(&testableSmsSender);  
    }  
  
public:  
    tm getTime(int year, int mon, int day, int hour, int min) {  
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };  
        mktime(&result);  
        return result;  
    }  
  
    tm plusHour(tm base, int hour) {  
        base.tm_hour += hour;  
        mktime(&base);  
        return base;  
    }  
  
    tm NOT_ON_THE_HOUR;  
    tm ON_THE_HOUR;  
    Customer CUSTOMER{ "Fake name", "010-1234-5678" };  
    const int UNDER_CAPACITY = 1;  
    const int CAPACITY_PER_HOUR = 3;  
  
    BookingScheduler bookingScheduler{ CAPACITY_PER_HOUR };  
    TestableSmsSender testableSmsSender;  
};
```

테스트 케이스 작성

✓ Step7-2. 완성된 코드 확인하기

- 아래 코드와 다름이 없음을 확인하기

```
TEST_F(BookingItem, 예약완료시_SMS는_무조건_발송) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, CAPACITY_PER_HOUR, CUSTOMER };  
  
    //act  
    bookingScheduler.addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(true, testableSmsSender.isSendMethodIsCalled());  
}
```

Step 7에서 작성한 테스트케이스

테스트 케이스 작성

✓ Step8-1. Email Test

- 고객이 Email 주소가 존재한다면, Email을 보내야한다.
- Unit Test를 할 때마다 Email을 보낼 수 없기에 대역 (Test Double) 이 필요하다.
- Test Double을 제작하기 위해,
먼저 MailSender를 Overriding 할 수 있도록 Virtual 키워드를 붙여준다.

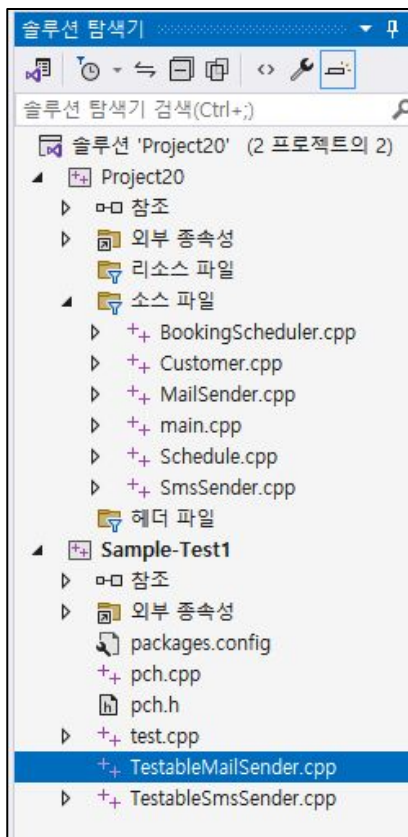
```
#pragma once
#include "Schedule.cpp"

class MailSender {
public:
    virtual void sendMail(Schedule *schedule) {
        //send mail
    }
};
```

테스트 케이스 작성

✓ Step8-2. Email Test

- Test Double용 새로운 Class를 생성한다.



TestableMailSender.cp

```
#include "pch.h"
#include "../Project20/MailSender.cpp"

class TestableMailSender : public MailSender {
public:
    void sendMail(Schedule* schedule) override {
        countSendMailMethodIsCalled++;
    }

    int getCountSendMailMethodIsCalled() {
        return countSendMailMethodIsCalled;
    }

private:
    int countSendMailMethodIsCalled = 0;
};
```

테스트 케이스 작성

✓ Step8-3. Email이 없을 때, 이메일 발송 여부 테스트

- 고객이 Email 주소가 없다면, Email을 보내면 안된다.
- 제작한 Test Double을 이용하여, send 메서드가 호출 되었는지 Behavior 검증을 한다.

```
#include "pch.h"
#include "../Project20/BookingScheduler.cpp"
#include "TestableSmsSender.cpp"
#include "TestableMailSender.cpp"
```

test.cpp 에 헤더파일 추가하기

```
TEST_F(BookingItem, 이메일이_없는_경우에는_이메일_미발송) {
    //arrange
    TestableMailSender testableMailSender;
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };
    bookingScheduler.setMailSender(&testableMailSender);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    EXPECT_EQ(0, testableMailSender.getCountSendMailMethodIsCalled());
}
```

CUSTOMER는 email이 없는 고객이다.

테스트 케이스 작성

✓ Step9. Email이 있을 때, 이메일 발송 여부 테스트

- 고객이 Email 주소가 있다면, Email을 보내야한다.
- 제작한 Test Double을 이용하여, send 메서드가 호출 되었는지 Behavior 검증을 한다.

```
TEST_F(BookingItem, 이메일이_있는_경우에는_이메일_발송) {  
    //arrange  
    Customer customerWithMail{ "Fake Name", "010-1234-5678", "test@test.com" };  
    TestableMailSender testableMailSender;  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, customerWithMail };  
    bookingScheduler.setMailSender(&testableMailSender);  
  
    //act  
    bookingScheduler.addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(1, testableMailSender.getCountSendMailMethodIsCalled());  
}
```

테스트 케이스 작성

✓ Step10-1. 중복 코드 리팩토링

- email Test Double을 사용한 테스트케이스들의 중복코드를 없애는 리팩토링 진행

```
class BookingItem : public testing::Test {
protected:
    void SetUp() override {
        NOT_ON_THE_HOUR = getTime(2021, 3, 26, 9, 5);
        ON_THE_HOUR = getTime(2021, 3, 26, 9, 0);

        bookingScheduler.setSmsSender(&testableSmsSender);
        bookingScheduler.setMailSender(&testableMailSender);
    }

public:
    tm getTime(int year, int mon, int day, int hour, int min) {
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };
        mktime(&result);
        return result;
    }

    tm plusHour(tm base, int hour) {
        base.tm_hour += hour;
        mktime(&base);
        return base;
    }

    tm NOT_ON_THE_HOUR;
    tm ON_THE_HOUR;
    Customer CUSTOMER{ "Fake name", "010-1234-5678" };
    Customer CUSTOMER_WITH_MAIL{ "Fake Name", "010-1234-5678", "test@test.com" };

    const int UNDER_CAPACITY = 1;
    const int CAPACITY_PER_HOUR = 3;

    BookingScheduler bookingScheduler{ CAPACITY_PER_HOUR };
    TestableSmsSender testableSmsSender;
    TestableMailSender testableMailSender;
};
```

테스트 케이스 작성

✓ Step10-2. 리팩토링 완료된, 테스트 코드

- Step 8, Step 9에서 작성된 테스트케이스

```
TEST_F(BookingItem, 이메일이_없는_경우에는_이메일_미발송) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };  
  
    //act  
    bookingScheduler.addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(0, testableMailSender.getCountSendMailMethodIsCalled());  
}  
  
TEST_F(BookingItem, 이메일이_있는_경우에는_이메일_발송) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL };  
  
    //act  
    bookingScheduler.addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(1, testableMailSender.getCountSendMailMethodIsCalled());  
}
```

테스트 케이스 작성

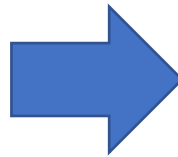
✓ Step11-1. 현재 Code Coverage 측정해보기

- (코드 커버리지 측정 명령어 예시)

- OpenCppCoverage.exe --sources C:*.c --export_type=html:coverage -- ./x64/Debug/Sample-Test1.exe

```
Git Bash
+ Git Bash - [icon] [icon] [icon]
테스트용 SmsSender class의 send메서드 실행됨
테스트용 SmsSender class의 send메서드 실행됨
[ OK ] BookingItem.시간대별 인원제한이 있다_같은_시간대가_다르면_Capacity_자있어도_스케줄_추가_성공 (0 ms)
[ RUN ] BookingItem.예약완료시 SMS는_무조건_발송
테스트용 SmsSender class의 send메서드 실행됨
[ OK ] BookingItem.예약완료시 SMS는_무조건_발송 (0 ms)
[ RUN ] BookingItem.이메일이_없는_경우에는_이메일_미발송
테스트용 SmsSender class의 send메서드 실행됨
[ OK ] BookingItem.이메일이_없는_경우에는_이메일_미발송 (0 ms)
[ RUN ] BookingItem.이메일이_있는_경우에는_이메일_발송
테스트용 SmsSender class의 send메서드 실행됨
[ OK ] BookingItem.이메일이_있는_경우에는_이메일_발송 (0 ms)
[ RUN ] BookingItem.현재날짜가_일요일인_경우_예약불가_예외처리
[ OK ] BookingItem.현재날짜가_일요일인_경우_예약불가_예외처리 (0 ms)
[ RUN ] BookingItem.현재날짜가_일요일이_아닌경우_예약가능
[ OK ] BookingItem.현재날짜가_일요일이_아닌경우_예약가능 (0 ms)
[-----] 9 tests from BookingItem (7 ms total)

[-----] Global test environment tear-down
[=====] 9 tests from 1 test case ran. (8 ms total)
[ PASSED ] 9 tests.
[info] Module: C:\Windows\System32\kernel.appcore.dll is selected because it matches selected pattern: *
[info] Module: C:\Windows\System32\msvcrt.dll is selected because it matches selected pattern: *
[info] -----
[info] Coverage generated in Folder C:\Users\inho\source\repos\Project20\coverage
[info] -----
[info] The code coverage report is not what you expect? See the FAQ https://github.com/OpenCppCoverage/OpenCppCoverage/wiki/FAQ.
inho@chayoonPC MINGW64 ~/source/repos/Project20
$
```



테스트 케이스 작성

✓ Step1 1-2. 일요일 예약시 Exception 코드, 주식 풀기

- 주석을 켜 후, 다시 Code Coverage를 측정해본다. (97%로 줄어들었다.)
- Code Coverage를 다시 100%를 만들어야 한다.



```
void addSchedule(Schedule* schedule) {

    // 정각에 예약하지 않을 경우 RuntimeException 발생
    if (schedule->getDateTime().tm_min != 0) {
        throw std::runtime_error("Booking should be on the hour.");
    }

    // 시간당 예약인원을 초과할 경우 RuntimeException 발생
    int numberOfPeople = schedule->getNumberOfPeople();
    for (Schedule* bookedSchedule : schedules) {
        if (isSameTime(bookedSchedule->getDateTime(), schedule->getDateTime())) {
            numberOfPeople += bookedSchedule->getNumberOfPeople();
        }
    }
    if (numberOfPeople > capacityPerHour) {
        throw std::runtime_error("Number of people is over restaurant capacity per hour");
    }

    // 일요일에는 시스템을 오픈하지 않는다.
    time_t now = time(nullptr);
    if (getDayOfWeek(now) == "Sunday") {
        throw std::runtime_error("Booking system is not available on sunday");
    }

    schedules.push_back(schedule);

    // 고객에게 SMS 발송
    smsSender->send(schedule);
}
```



주석풀기

```

1. #pragma once
2. #include <vector>
3. #include <stdexcept>
4. #include "algorithm"
5. #include "Schedule.hpp"
6. #include "MailSender.hpp"
7. #include "SmsSender.hpp"
8.
9. using std::vector;
10.
11. class BookingScheduler {
12. public:
13.     BookingScheduler(int capacityPerHour) :
14.         capacityPerHour{ capacityPerHour } {
15.         smsSender = new SmsSender();
16.         mailSender = new MailSender();
17.     }
18.
19.     void addSchedule(Schedule* schedule) {
20.
21.         // Áð· ¢¿ ¿ ¿' %ÀÁÃÄ Å Æ· ° æ ¿ i RuntimeException ' ß·ý
22.         if (schedule->getDateTime().tm_min != 0) {
23.             throw std::runtime_error("Booking should be on the hour.");
24.         }
25.
26.         // ½· ¤· ¿ ¿' %ÀÁÃÄ Å Æ· ° æ ¿ i RuntimeException ' ß·ý
27.         int numberOfPeople = schedule->getNumberOfPeople();
28.         for (Schedule* bookedSchedule : schedules) {
29.             if (isSameTime(bookedSchedule->getDateTime(), schedule->getDateTime())) {
30.                 numberOfPeople += bookedSchedule->getNumberOfPeople();
31.             }
32.         }
33.         if (numberOfPeople > capacityPerHour) {
34.             throw std::runtime_error("Number of people is over restaurant capacity per hour");
35.         }
36.
37.         // Á¿ ÄÁ¿ ¿ · Å ½· ¤ ÁÒÁ· ¿ ÀÁÁÃÄ Å Æ· Å · Ö.
38.         time_t now = time(nullptr);
39.         if (getDayOfWeek(now) == "Sunday") {
40.             throw std::runtime_error("Booking system is not available on sunday");
41.         }
42.
43.         schedules.push_back(schedule);
44.
45.         // ° ° · ¿ ¿ ° SMS · ß ¼ Ö
46.         smsSender->send(schedule);
47.         // ° ° · Á¿ E Mail· ° ¿ Áö° ¿ ÁÒÁ· ° æ ¿ i E Mail · ß ¼ Ö
48.         if (schedule->getCustomer()->getEmail() != "") {

```

테스트 케이스 작성

✓ Step11-3. 대역 필요성, 이해하기

- 오늘 (PC 기준) 날짜의 정보를 가져온다.
- 오늘 날짜가 '일요일'이라면, Exception 처리한다.

✓ Exception이 잘 되는지 테스트 하려면, 일요일에만 테스트가 가능하다.

항시 테스트 하기 위해서는? □ **대역 필요 (Test Double)**

- 무조건 일요일로 인식하는 대역(가짜) Class 필요

이 부분이 다른 코드로
동작되는 가짜 Class를 만들자.

```
// 일요일에는 시스템을 오픈하지 않는다.  
time_t now = time(nullptr);  
if (getDayOfWeek(now) == "Sunday") {  
    throw std::runtime_error("Booking system is not available on sunday");  
}
```

테스트 케이스 작성

✓ Step11-4. Mocking을 위한 리팩토링

- `getNow()` 함수로 Extract Method를 한다.
- 대역 Class가 `getNow` 메서드를 Overriding 하고, 다른 날짜를 return하도록 처리예정

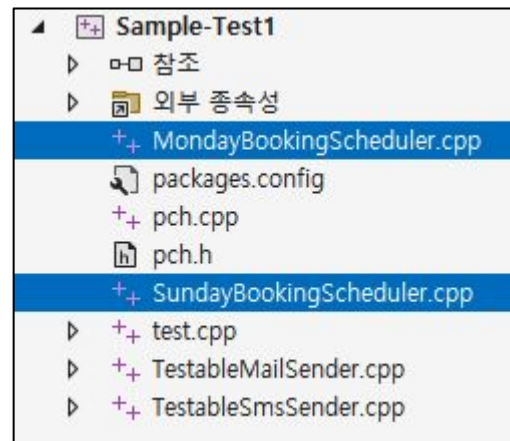
```
// 일요일에는 시스템을 오픈하지 않는다.  
time_t now = getNow();  
if (getDayOfWeek(now) == "Sunday") {  
    throw std::runtime_error("Booking system  
}  
  
schedules.push_back(schedule);  
  
// 고객에게 SMS 발송  
smsSender->send(schedule);  
// 고객이 E Mail을 가지고 있을 경우 E Mail 발송  
if (schedule->getCustomer().getEmail() != "")  
    mailSender->sendMail(schedule);  
}  
  
virtual time_t getNow() {  
    return time(nullptr);  
}
```

virtual 키워드로
함수 추가 제작

테스트 케이스 작성

✓ Step11-5. 대역 Class 2개 생성하기

- BookingScheduler.cpp 의 대역 Class 2개 생성
 - SundayBookingScheduler.cpp : getNow() 메서드는 무조건 일요일로 리턴한다.
 - MondyBookingScheduler.cpp : getNow() 메서드는 무조건 월요일로 리턴한다.



SundayBookingScheduler.cpp

```
#include "pch.h"
#include "../Project20/BookingScheduler.cpp"

class SundayBookingScheduler : public BookingScheduler {
public:
    SundayBookingScheduler(int capacityPerHour) :
        BookingScheduler{ capacityPerHour } {

    }

    time_t getNow() override {
        return getTime(2021, 3, 28, 17, 0);
    }

private:
    time_t getTime(int year, int mon, int day, int hour, int min) {
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };
        return mktime(&result);
    }
};
```

MondayBookingScheduler.cpp

```
#include "pch.h"
#include "../Project20/BookingScheduler.cpp"

class MondayBookingScheduler : public BookingScheduler {
public:
    MondayBookingScheduler(int capacityPerHour) :
        BookingScheduler{ capacityPerHour } {

    }

    time_t getNow() override {
        return getTime(2024, 6, 3, 17, 0);
    }

private:
    time_t getTime(int year, int mon, int day, int hour, int min) {
        tm result = { 0, min, hour, day, mon - 1, year - 1900, 0, 0, -1 };
        return mktime(&result);
    }
};
```

테스트 케이스 작성

✓ Step11-6. 일요일 Mocking을 사용하는 테스트 케이스 만들기

- 일요일 예약시, 정상적으로 Exception 발생이 되는지 테스트

```
#include "pch.h"
#include "../Project20/BookingScheduler.cpp"
#include "TestableSmsSender.cpp"
#include "TestableMailSender.cpp"
#include "SundayBookingScheduler.cpp"
#include "MondayBookingScheduler.cpp"
```

test.cpp에 헤더파일 추가

```
TEST_F(BookingItem, 현재날짜가_일요일인_경우_예약불가_예외처리) {
    //arrange
    BookingScheduler *bookingScheduler = new SundayBookingScheduler{CAPACITY_PER_HOUR};

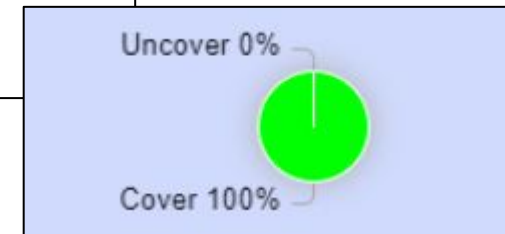
    try {
        //act
        Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL };
        bookingScheduler->addSchedule(schedule);
        FAIL();
    }
    catch (std::runtime_error &e) {
        //assert
        EXPECT_EQ(string{ e.what() }, string{ "Booking system is not available on sunday" });
    }
}
```

테스트 케이스 작성

✓ Step11-7. 월요일 Mocking을 사용하는 테스트 케이스 만들기

- 월요일 예약시, 정상적으로 예약이 되는지 확인하는 테스트 케이스
- 코드 커버리지가 100% 되는지 확인한다.

```
TEST_F(BookingItem, 현재날짜가_일요일이_아닌경우_예약가능) {  
    //arrange  
    BookingScheduler* bookingScheduler = new MondayBookingScheduler{ CAPACITY_PER_HOUR };  
  
    //act  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL };  
    bookingScheduler->addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(true, bookingScheduler->hasSchedule(schedule));  
}
```



테스트 케이스 작성

✓ Step12. 통합 Test Double 클래스 만들기

- SundayBookingScheduler, MondayBookingScheduler를 통합한 TestableScheduler 클래스 만들기

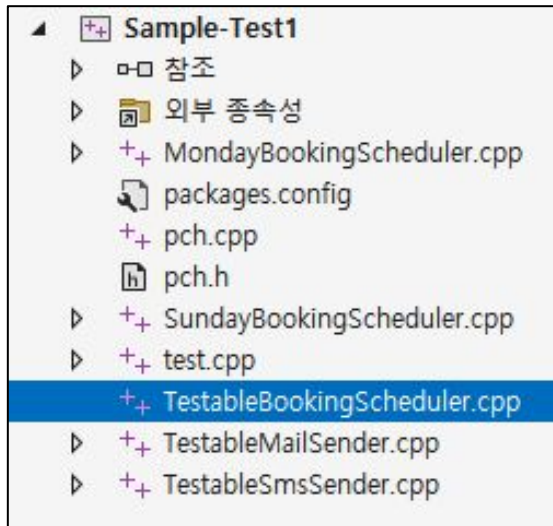
✓ 진행순서

1. TestableScheduler 클래스 만들기
2. 기존 테스트 코드를 TestableScheduler 클래스를 사용하도록 변경
3. SundayBookingScheduler, MondayBookingScheduler 클래스를 삭제

테스트 케이스 작성

✓ Step12-1. TestableScheduler 클래스 만들기

- 원하는 날짜를 객체 생성시 지정할 수 있는 Test Double 클래스 제작



```
#include "pch.h"
#include "../Project20/BookingScheduler.cpp"

class TestableBookingScheduler : public BookingScheduler {
public:
    TestableBookingScheduler(int capacityPerHour, tm dateTime) :
        BookingScheduler{ capacityPerHour },
        dateTime{ dateTime } {

    }

    time_t getNow() override {
        return mktime(&dateTime);
    }

private:
    tm dateTime;
};
```

TestableBookingScheduler.cpp

[도전] 테스트 케이스 작성

✓ Step12-2. 통합 Test Double 클래스로 대체하기 **Mission !**

- Step 11에서 만들었던 테스트케이스를 통합 Test Double 클래스로 대체하기
- 대체가 완료되었다면,
기존 SundayBookingScheduler, MondayBookingScheduler 클래스를 삭제 한다.

```
TEST_F(BookingItem, 현재날짜가_일요일인_경우_예약불가_예외처리) {  
    //arrange  
    BookingScheduler* bookingScheduler = new SundayBookingScheduler{CAPACITY_PER_HOUR};  
  
    try {  
        //act  
        Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL };  
        bookingScheduler->addSchedule(schedule);  
        FAIL();  
    }  
    catch (std::runtime_error &e) {  
        //assert  
        EXPECT_EQ(string{ e.what() }, string{ "Booking system is not available on sunday" });  
    }  
}  
  
TEST_F(BookingItem, 현재날짜가_일요일이_아닌경우_예약가능) {  
    //arrange  
    BookingScheduler* bookingScheduler = new MondayBookingScheduler{ CAPACITY_PER_HOUR };  
  
    //act  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL };  
    bookingScheduler->addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(true, bookingScheduler->hasSchedule(schedule));  
}
```

Step 11의 두 테스트케이스를 리팩토링 한다.

gMock을 사용한 유닛테스트 리팩토링

2 단계 : 테스트 케이스 리팩토링 (with gmock)

Customer를 Mock으로 대체

먼저, gMock Library를 설치한다.
설치 방법은 생략.

✓ Step 1-1 : Customer을 Mock 객체로 변경

실제 고객 정보를 입력할 필요가 없기에, Mock 객체로 대체

```
test.cpp

#include "gtest/gtest.h"
#include "gmock/gmock.h"
#include "../Project2/BookingScheduler.cpp"
#include "TestableSmsSender.cpp"
#include "TestableMailSender.cpp"
#include "TestableBookingScheduler.cpp"

class MockCustomer : public Customer {
public:
    MOCK_METHOD(string, getEmail, (), (override));
};
```

Customer 대신 Mock Object를 사용할 준비

```
Customer.cpp

class Customer {
public:
    Customer(string name, string phoneNumber) :
        name{ name },
        phoneNumber{ phoneNumber } {
    }

    Customer(string name, string phoneNumber, string email) :
        name{ name },
        phoneNumber{ phoneNumber },
        email{ email } {
    }

    virtual string getEmail() {
        return email;
    }
};
```

Virtual 키워드 추가

Customer를 Mock으로 대체

✓ Step 1-2 : Customer을 Mock 객체로 변경

test.cpp

```
class BookingItem : public testing::Test {  
protected:
```

```
    //...
```

```
public:
```

```
    //...
```

```
    MockCustomer CUSTOMER;
```

```
    MockCustomer CUSTOMER_WITH_MAIL;
```

```
    //Customer CUSTOMER{ "Fake name", "010-1234-5678" };
```

```
    //Customer CUSTOMER_WITH_MAIL{ "Fake Name", "010-1234-5678", "test@test.com" };
```

```
    const int UNDER_CAPACITY = 1;
```

```
    const int CAPACITY_PER_HOUR = 3;
```

Customer를 Mock으로 대체

✓ Step 1-3 : Customer을 Mock 객체로 변경

- CUSTOMER_WITH_MAIL 도 똑같이 적용한다.
- Stub 을 다르게 걸어준다.

```
class BookingItem : public testing::Test {
protected:
    void SetUp() override {
        NOT_ON_THE_HOUR = getTime(2021, 3, 26, 9, 5);
        ON_THE_HOUR = getTime(2021, 3, 26, 9, 0);
        SUNDAY_ON_THE_HOUR = getTime(2021, 3, 28, 17, 0);
        MONDAY_ON_THE_HOUR = getTime(2024, 6, 3, 17, 0);
        bookingScheduler.setSmsSender(&testableSmsSender);
        bookingScheduler.setMailSender(&testableMailSender);

        EXPECT_CALL(CUSTOMER, getEmail, (), ())
            .WillRepeatedly(testing::Return(""));
        EXPECT_CALL(CUSTOMER_WITH_MAIL, getEmail, (), ())
            .WillRepeatedly(testing::Return("test@test.com"));
    }

public:
    tm getTime(int year, int mon, int day, int hour, int min) {
```

test.cpp

Customer를 Mock으로 대체

✓ Step 1-4 : Schedule.cpp 수정

- 목 객체의 참조를 넣어줄 수 있게끔 복사 가 아닌 레퍼런스로 변경.

Schedule.cpp

```
class Schedule {
public:
    Schedule(tm dateTime, int numberOfPeople, Customer &customer) :
        dateTime{ dateTime },
        numberOfPeople{ numberOfPeople },
        customer{ customer } {
    }

    tm getDateTime() {
        return dateTime;
    }

    int getNumberOfPeople() {
        return numberOfPeople;
    }

    Customer& getCustomer() {
        return customer;
    }

private:
    tm dateTime;
    int numberOfPeople;
    Customer &customer;
};
```

Customer를 Mock으로 대체

✓ Step 1-5 : 결과 확인

- 변경을 했을 때 테스트가 통과해야 한다.

```
Microsoft Visual Studio 디버그 콘솔

Running main() from gmock_main.cc
----- Running 9 tests from 1 test suite.
Global test environment set-up.
----- 9 tests from BookingItem
RUN OK BookingItem.예약은 정시에만 가능하다.정시가 아닌 경우 예약불가 (0 ms)
RUN OK BookingItem.예약은 정시에만 가능하다.정시가 아닌 경우 예약불가 (0 ms)
BookingItem.예약은 정시에만 가능하다.정시인 경우 예약가능
테스트용 SmsSender class의 send메서드가 실행됨
RUN OK BookingItem.예약은 정시에만 가능하다.정시인 경우 예약가능 (0 ms)
BookingItem.시간대별 인원 제한이 있다.같은 시간대에 Capacity 초과할 경우 예외발생
테스트용 SmsSender class의 send메서드가 실행됨
RUN OK BookingItem.시간대별 인원 제한이 있다.같은 시간대에 Capacity 초과할 경우 예외발생 (0 ms)
BookingItem.시간대별 인원 제한이 있다.같은 시간대가 다르면 Capacity 차있어도 스케줄 추가 성공
테스트용 SmsSender class의 send메서드가 실행됨
테스트용 SmsSender class의 send메서드가 실행됨
RUN OK BookingItem.시간대별 인원 제한이 있다.같은 시간대가 다르면 Capacity 차있어도 스케줄 추가 성공 (0 ms)
BookingItem.예약완료시 SMS는 무조건 발송
테스트용 SmsSender class의 send메서드가 실행됨
RUN OK BookingItem.예약완료시 SMS는 무조건 발송 (0 ms)
BookingItem.이메일이 없는 경우에는 이메일 미발송
테스트용 SmsSender class의 send메서드가 실행됨
RUN OK BookingItem.이메일이 없는 경우에는 이메일 미발송 (0 ms)
BookingItem.이메일이 있는 경우에는 이메일 발송
테스트용 SmsSender class의 send메서드가 실행됨
RUN OK BookingItem.이메일이 있는 경우에는 이메일 발송 (0 ms)
BookingItem.현재날짜가 월요일인 경우 예약불가 예외처리
RUN OK BookingItem.현재날짜가 월요일인 경우 예약불가 예외처리 (0 ms)
BookingItem.현재날짜가 월요일이 아닌 경우 예약가능
RUN OK BookingItem.현재날짜가 월요일이 아닌 경우 예약가능 (0 ms)
----- 9 tests from BookingItem (3 ms total)

----- Global test environment tear-down
----- 9 tests from 1 test suite ran. (6 ms total)
PASSED 9 tests.
```

TestableSender를 Mock 객체로 대체

✓ Step 2 : TestableSmsSender / TestableMailSender를 Mock 객체로 대체

- SmsSender, MailSender을 상속해서 만든 Test Double
- 다음 페이지부터, 이 객체를 gMock을 이용한 객체로 변경할 예정

test.cpp

```
class BookingItem : public testing::Test {
    // ...
public:
    void SetUp() override {
        //...
        bookingScheduler.setSmsSender(&testableSmsSender);
        bookingScheduler.setMailSender(&testableMailSender);
        // ...
    }

    BookingItem() :
        mockCustomer{ },
        mockCustomerWithMail{ },
        CUSTOMER{ mockCustomer },
        CUSTOMER_WITH_MAIL{ mockCustomerWithMail }
    {
    }

    //...
    TestableSmsSender testableSmsSender;
    TestableMailSender testableMailSender;
    //...
```

TestableSender를 Mock 객체로 대체

✓ Step 2-1 : TestableSmsSender 를 Mock 객체로 대체

- 직접 만든 Test Double 객체 생성 대신, gMock을 이용해 변경한다.

TestableSmsSender.cpp

```
#include <iostream>
#include "../Project2/SmsSender.cpp"

class TestableSmsSender : public SmsSender {
public:
    void send(Schedule* schedule) override {
        std::cout << "테스트용 SmsSender class의 send메서드 실행됨\n";
        sendMethodIsCalled = true;
    }

    bool isSendMethodIsCalled() {
        return sendMethodIsCalled;
    }

private:
    bool sendMethodIsCalled;
};
```



```
#include "gtest/gtest.h"
#include "gmock/gmock.h"
#include "../Project2/SmsSender.cpp"

class TestableSmsSender : public SmsSender {
public:
    MOCK_METHOD(void, send, (Schedule*), (override));
};
```

TestableSender를 Mock 객체로 대체

✓ Step 2-2 : TestableMailSender를 Mock 객체로 대체

- 직접 만든 Test Double 객체 생성 대신, Mock 객체 생성으로 변경한다.

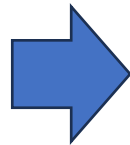
TestableMailSender.cpp

```
#include "../Project2/MailSender.cpp"

class TestableMailSender : public MailSender {
public:
    void sendMail(Schedule* schedule) override {
        countSendMailMethodIsCalled++;
    }

    int getCountSendMailMethodIsCalled() {
        return countSendMailMethodIsCalled;
    }

private:
    int countSendMailMethodIsCalled = 0;
};
```



```
#include "gtest/gtest.h"
#include "gmock/gmock.h"
#include "../Project2/MailSender.cpp"

class TestableMailSender : public MailSender {
public:
    MOCK_METHOD(void, sendMail, (Schedule*), (override));
};
```

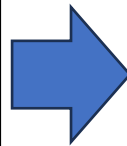
TestableSender를 Mock 객체로 대체

✓ Step 2-3 : NiceMock 으로 변경

- Stub을 하지 않더라도 WARNING이 뜨지 않도록 NiceMock 사용

test.cpp

```
//Customer CUSTOMER{ "Fake Name", "010-1234-5678" },  
//Customer CUSTOMER_WITH_MAIL{ "Fake Name", "010-1234-5678", "test@test.com" };  
  
const int UNDER_CAPACITY = 1;  
const int CAPACITY_PER_HOUR = 3;  
  
BookingScheduler bookingScheduler{ CAPACITY_PER_HOUR };  
TestableSmsSender testableSmsSender;  
TestableMailSender testableMailSender;  
};
```



```
//Customer CUSTOMER{ "Fake Name", "010-1234-5678" },  
//Customer CUSTOMER_WITH_MAIL{ "Fake Name", "010-1234-5678", "test@test.com" };  
  
const int UNDER_CAPACITY = 1;  
const int CAPACITY_PER_HOUR = 3;  
  
BookingScheduler bookingScheduler{ CAPACITY_PER_HOUR };  
testing::NiceMock<TestableSmsSender> testableSmsSender;  
testing::NiceMock<TestableMailSender> testableMailSender;  
};
```

TestableSender를 Mock 객체로 대체

✓ Step 2-4 : Mock 객체 행위 검증 (SmsSender)

- send 메서드 1회 호출되어야 Pass

```
TEST_F(BookingItem, 예약완료시_SMS는_무조건_발송) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, CAPACITY_PER_HOUR, CUSTOMER };  
  
    //act  
    bookingScheduler.addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(true, testableSmsSender.isSendMethodIsCalled());  
}
```



```
TEST_F(BookingItem, 예약완료시_SMS는_무조건_발송) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, CAPACITY_PER_HOUR, CUSTOMER };  
  
    //act, assert  
    EXPECT_CALL(testableSmsSender, send(schedule))  
        .Times(1);  
  
    bookingScheduler.addSchedule(schedule);  
}
```

TestableSender를 Mock 객체로 대체

✓ Step 2-5 : Mock 객체 행위 검증 (MailSender)

- sendMail 메서드가 호출된 적이 없어야 Pass

```
TEST_F(BookingItem, 이메일이_없는_경우에는_이메일_미발송) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };  
  
    //act  
    bookingScheduler.addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(0, testableMailSender.getCountSendMailMethodIsCalled());  
}
```



```
TEST_F(BookingItem, 이메일이_없는_경우에는_이메일_미발송) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER };  
  
    //act, assert  
    EXPECT_CALL(testableMailSender, sendMail(schedule))  
        .Times(0);  
  
    bookingScheduler.addSchedule(schedule);  
}
```

TestableSender를 Mock 객체로 대체

✓ Step 2-6 : Mock 객체 행위 검증 (MailSender)

- sendMail 메서드 1회 호출되어야 Pass

```
TEST_F(BookingItem, 이메일이_있는_경우에는_이메일_발송) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL };  
  
    //act  
    bookingScheduler.addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(1, testableMailSender.getCountSendMailMethodIsCalled());  
}
```



```
TEST_F(BookingItem, 이메일이_있는_경우에는_이메일_발송) {  
    //arrange  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL };  
  
    //act, assert  
    EXPECT_CALL(testableMailSender, sendMail(schedule))  
        .Times(1);  
    bookingScheduler.addSchedule(schedule);  
}
```

TestableSender를 Mock 객체로 대체

✓ Step 2-7 : 결과 확인

- 테스트가 모두 통과해야 한다.

```
Microsoft Visual Studio 디버그 콘솔
Running main() from gmock_main.cc
Running 9 tests from 1 test suite.
Global test environment set-up.
9 tests from BookingItem
[ RUN      ] BookingItem.예약은 정시에만 가능하다.정시가 아닌경우 예약불가
[ OK       ] BookingItem.예약은 정시에만 가능하다.정시가 아닌경우 예약불가 (0 ms)
[ RUN      ] BookingItem.예약은 정시에만 가능하다.정시인 경우 예약가능
[ OK       ] BookingItem.예약은 정시에만 가능하다.정시인 경우 예약가능 (0 ms)
[ RUN      ] BookingItem.시간대별 인원제한이 있다.같은 시간대에 Capacity_초과할 경우 예외발생
[ OK       ] BookingItem.시간대별 인원제한이 있다.같은 시간대에 Capacity_초과할 경우 예외발생 (0 ms)
[ RUN      ] BookingItem.시간대별 인원제한이 있다.같은 시간대가 다르면 Capacity_차이에도 스케줄 추가 성공
[ OK       ] BookingItem.시간대별 인원제한이 있다.같은 시간대가 다르면 Capacity_차이에도 스케줄 추가 성공 (0 ms)
[ RUN      ] BookingItem.예약완료시 SMS는 무조건 발송
[ OK       ] BookingItem.예약완료시 SMS는 무조건 발송 (0 ms)
[ RUN      ] BookingItem.이메일이 없는 경우에는 이메일 미발송
[ OK       ] BookingItem.이메일이 없는 경우에는 이메일 미발송 (0 ms)
[ RUN      ] BookingItem.이메일이 있는 경우에는 이메일 발송
[ OK       ] BookingItem.이메일이 있는 경우에는 이메일 발송 (0 ms)
[ RUN      ] BookingItem.현재날짜가 일요일인 경우 예약불가 예외처리
[ OK       ] BookingItem.현재날짜가 일요일인 경우 예약불가 예외처리 (0 ms)
[ RUN      ] BookingItem.현재날짜가 일요일이 아닌경우 예약가능
[ OK       ] BookingItem.현재날짜가 일요일이 아닌경우 예약가능 (0 ms)
----- 9 tests from BookingItem (2 ms total)

----- Global test environment tear-down
===== 9 tests from 1 test suite ran. (3 ms total)
[ PASSED ] 9 tests.
```

TestableBookingScheduler를 Mock 객체로 대체

✓ Step 3 : TestableBookingScheduler 대신 Mock 객체로 대체

- 현재 PC 날짜를 일요일인것 처럼 만들어주기 위한 Test Double
- 다음 페이지부터 아래 객체를, Stub 을 사용하는 객체로 대체할 예정

```
TEST_F(BookingItem, 현재날짜가_일요일인_경우_예약불가_예외처리) {  
    //arrange  
    BookingScheduler* bookingScheduler  
        = new TestableBookingScheduler{ CAPACITY_PER_HOUR, SUNDAY_ON_THE_HOUR };  
    try {  
        //act  
        Schedule* bookingSch  
        bookingSch  
        FAIL();  
    }  
    catch (std::ru  
        //assert  
        EXPECT_EQ(  
    }  
}  
  
TEST_F(BookingItem, 현재날짜가_일요일이_아닌경우_예약가능) {  
    //arrange  
    BookingScheduler* bookingScheduler  
        = new TestableBookingScheduler{ CAPACITY_PER_HOUR, MONDAY_ON_THE_HOUR };  
    //act  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL };  
    bookingScheduler->addSchedule(schedule);  
    //assert  
    EXPECT_EQ(true, bookingScheduler->hasSchedule(schedule));  
}
```

TestableBookingScheduler를 Mock 객체로 대체

✓ Step 3-1 : TestableBookingScheduler 대신 Mock 객체로 대체

- bookingScheduler 객체를 Mock 객체로 대체

TestableBookingScheduler.cpp

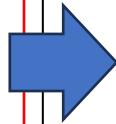
```
#include "../Project2/BookingScheduler.cpp"

class TestableBookingScheduler : public BookingScheduler {
public:
    TestableBookingScheduler(int capacityPerHour, tm dateTime) :
        BookingScheduler{ capacityPerHour },
        dateTime{ dateTime } {

    }

    time_t getNow() override {
        return mktime(&dateTime);
    }

private:
    tm dateTime;
};
```



```
#include "gtest/gtest.h"
#include "gmock/gmock.h"
#include "../Project2/BookingScheduler.cpp"

class TestableBookingScheduler : public BookingScheduler {
public:
    TestableBookingScheduler(int capacityPerHour) :
        BookingScheduler{ capacityPerHour } {

    }

    MOCK_METHOD(time_t, getNow, (), (override));
};
```

TestableBookingScheduler를 Mock 객체로 대체

✓ Step 3-2 : TestableBookingScheduler 대신 Mock 객체로 대체

- getNow() 메서드 호출시 일요일로 리턴하도록 Stubbing

test.cpp

```
TEST_F(BookingItem, 현재날짜가_일요일인_경우_예약불가_예외처리) {  
    //arrange  
    TestableBookingScheduler mockScheduler{CAPACITY_PER_HOUR};  
    EXPECT_CALL(mockScheduler, getNow())  
        .WillRepeatedly(testing::Return(mktime(&SUNDAY_ON_THE_HOUR)));  
  
    BookingScheduler* bookingScheduler = &mockScheduler;  
  
    try {  
        //act  
        Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL };  
        bookingScheduler->addSchedule(schedule);  
        FAIL();  
    }  
    catch (std::runtime_error& e) {  
        //assert  
        EXPECT_EQ(string{ e.what() }, string{ "Booking system is not available on sunday" });  
    }  
}
```

TestableBookingScheduler를 Mock 객체로 대체

✓ Step 3-3 : TestableBookingScheduler 대신 Mock 객체로 대체

- `getNow()` 메서드 호출시 월요일로 리턴하도록 Stubbing

```
TEST_F(BookingItem, 현재날짜가_일요일이_아닌경우_예약가능) {  
    //arrange  
    TestableBookingScheduler mockScheduler{ CAPACITY_PER_HOUR };  
    EXPECT_CALL(mockScheduler, getNow())  
        .WillRepeatedly(testing::Return(mktime(&MONDAY_ON_THE_HOUR)));  
  
    BookingScheduler* bookingScheduler = &mockScheduler;  
  
    //act  
    Schedule* schedule = new Schedule{ ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL };  
    bookingScheduler->addSchedule(schedule);  
  
    //assert  
    EXPECT_EQ(true, bookingScheduler->hasSchedule(schedule));  
}
```

TestableBookingScheduler를 Mock 객체로 대체

✓ Step 3-4 : 결과 확인

- 변경을 했을 때 테스트가 통과해야 한다.



```
Microsoft Visual Studio 디버그 콘솔
Running main() from gmock_main.cc
----- Running 9 tests from 1 test suite.
Global test environment set-up.
9 tests from BookingItem
[ RUN      ] BookingItem.예약은 정시에만 가능하다 정시가 아닌 경우 예약불가
[ OK       ] BookingItem.예약은 정시에만 가능하다 정시가 아닌 경우 예약불가 (0 ms)
[ RUN      ] BookingItem.예약은 정시에만 가능하다 정시인 경우 예약가능
[ OK       ] BookingItem.예약은 정시에만 가능하다 정시인 경우 예약가능 (0 ms)
[ RUN      ] BookingItem.시간대별 인원제한이 있다 같은 시간대에 Capacity 초과할 경우 예외발생
[ OK       ] BookingItem.시간대별 인원제한이 있다 같은 시간대에 Capacity 초과할 경우 예외발생 (0 ms)
[ RUN      ] BookingItem.시간대별 인원제한이 있다 같은 시간대가 다르다면 Capacity 차있어도 스케줄 추가 성공
[ OK       ] BookingItem.시간대별 인원제한이 있다 같은 시간대가 다르다면 Capacity 차있어도 스케줄 추가 성공 (0 ms)
[ RUN      ] BookingItem.예약완료시 SMS는 무조건 발송
[ OK       ] BookingItem.예약완료시 SMS는 무조건 발송 (0 ms)
[ RUN      ] BookingItem.이메일이 없는 경우에는 이메일 미발송
[ OK       ] BookingItem.이메일이 없는 경우에는 이메일 미발송 (0 ms)
[ RUN      ] BookingItem.이메일이 있는 경우에는 이메일 발송
[ OK       ] BookingItem.이메일이 있는 경우에는 이메일 발송 (0 ms)
[ RUN      ] BookingItem.현재날짜가 일요일인 경우 예약불가 예외처리
[ OK       ] BookingItem.현재날짜가 일요일인 경우 예약불가 예외처리 (0 ms)
[ RUN      ] BookingItem.현재날짜가 일요일이 아닌 경우 예약가능
[ OK       ] BookingItem.현재날짜가 일요일이 아닌 경우 예약가능 (0 ms)
----- 9 tests from BookingItem (5 ms total)

----- Global test environment tear-down
===== 9 tests from 1 test suite ran. (5 ms total)
[ PASSED ] 9 tests.
```